Practical 1: Implementation of Classical Cryptographic Ciphers

1. Aim

To implement Caesar, Vigenère, and Rail Fence ciphers for encryption and decryption of the plaintext **"HELLO WORLD"** and compare their working and effectiveness.

2. Software Required

- CrypTool 2
- Windows operating system

3. Theory

Classical cryptography uses simple mathematical operations to convert plaintext into ciphertext.

Caesar Cipher: A substitution cipher in which each letter is shifted by a fixed number of positions in the alphabet.

Vigenère Cipher: A polyalphabetic substitution cipher that uses a keyword to apply different shifts to the plaintext.

Rail Fence Cipher: A transposition cipher that rearranges plaintext characters in a zigzag pattern across multiple rows.

4. Algorithm

A. Caesar Cipher

1. Start.
2. Enter the plaintext HELLO WORLD.
3. Select a shift value of 3.
4. Replace each letter with the letter three positions ahead.
5. Display the encrypted text.
6. Reverse the shift to decrypt the ciphertext.
7. Stop.

B. Vigenère Cipher

1. Start.
2. Enter the plaintext HELLO WORLD.
3. Choose the keyword KEY.

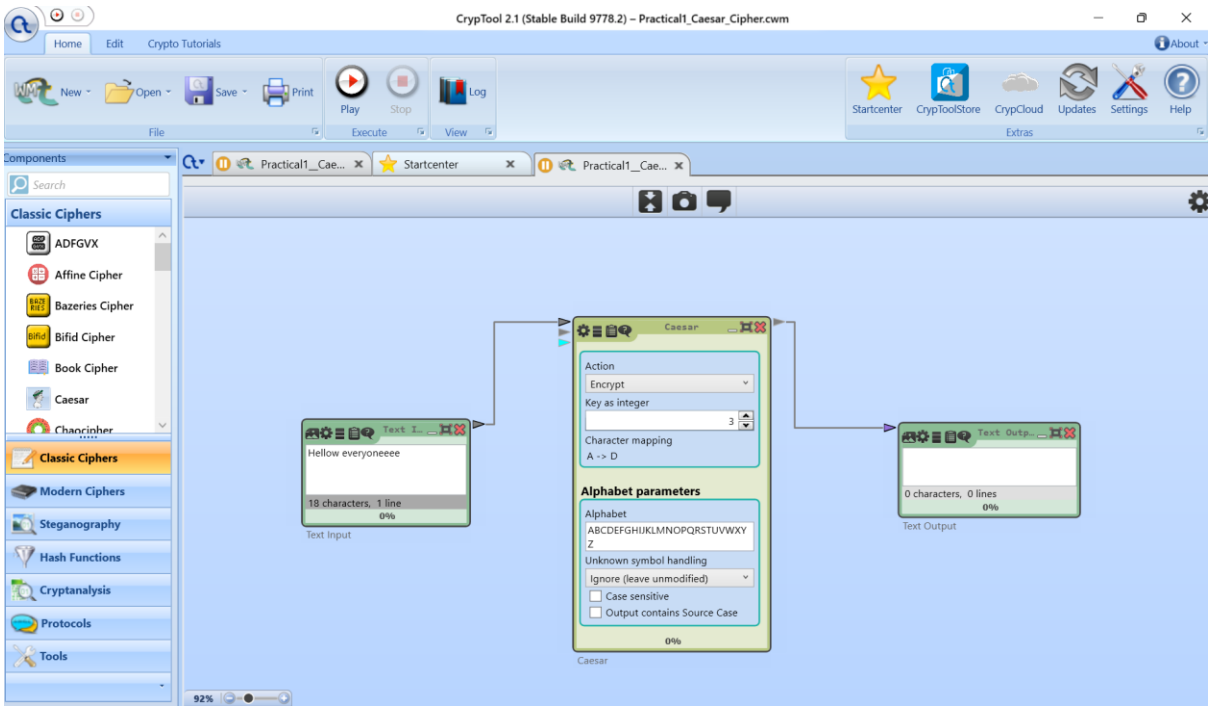
4. Repeat the keyword to match the number of letters.
5. Encrypt each letter using its corresponding keyword letter.
6. Decrypt using the same keyword.
7. Stop.

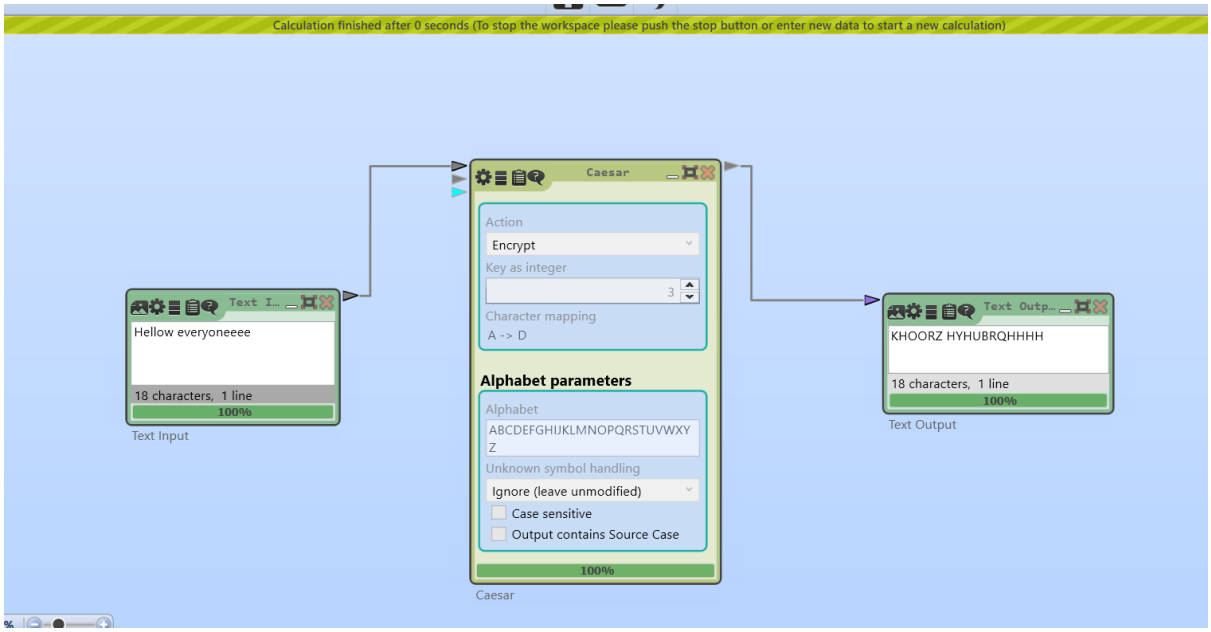
5. Working Procedure

Part A: Caesar Cipher Using CrypTool 2

1. Open **CrypTool 2**.
2. Open the file Practical1_Caesar_Cipher.cwm from your uploaded ZIP.
3. Locate the plaintext input component.
4. Enter HELLO WORLD.
5. Set the Caesar shift value to **3**.
6. Connect the components for encryption and decryption, if they are not already connected.
7. Click **Play** to execute the workflow.
8. Observe the encrypted and decrypted messages.

output:

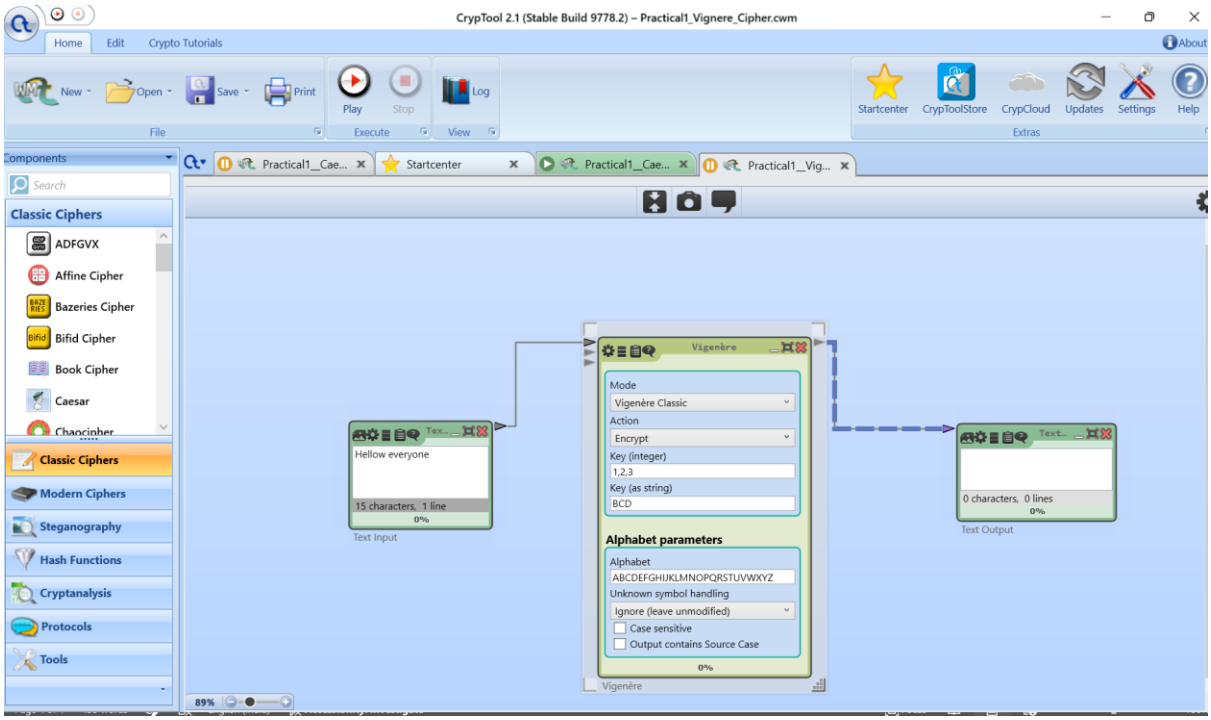


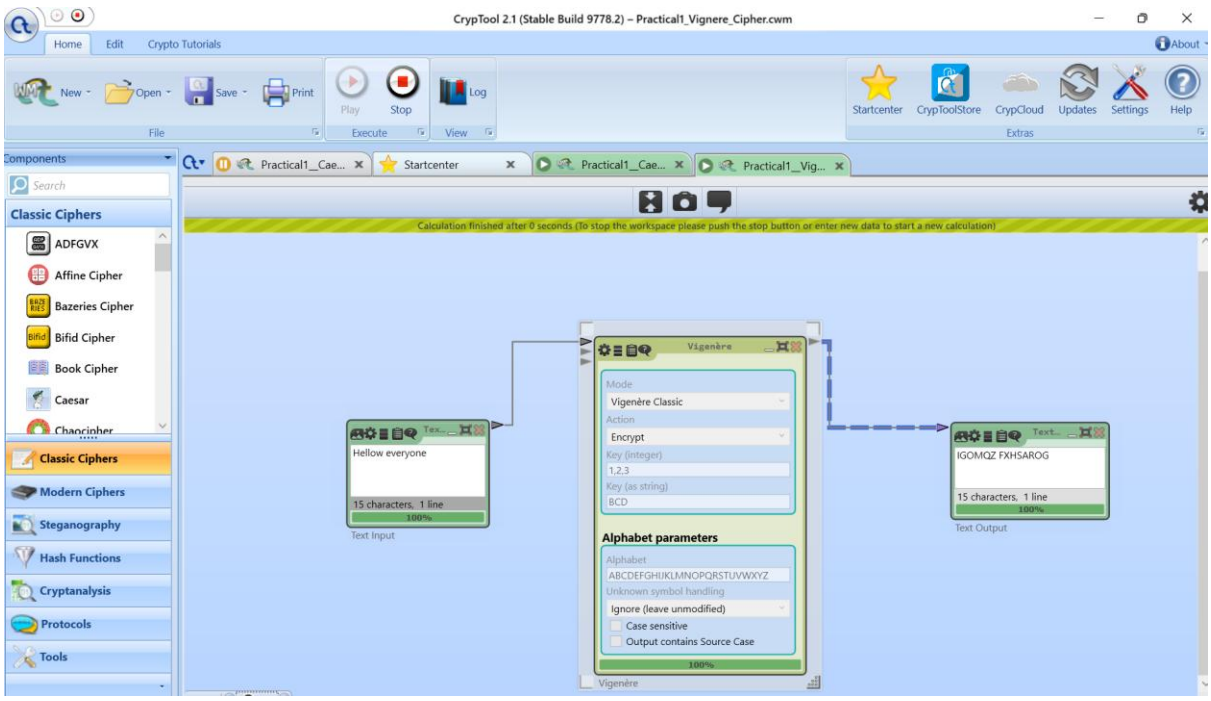


Part B: Vigenère Cipher Using CrypTool 2

1. Open Practical1_Vignere_Cipher.cwm in CrypTool 2.
2. Enter the plaintext HELLO WORLD.
3. Set the encryption key to KEY.
4. Execute the encryption workflow.
5. Observe the generated ciphertext.
6. Decrypt the ciphertext using the same key.
7. Verify that the original plaintext is recovered.

output:


CrypTool 2.1 (Stable Build 9778.2) – Practical1_Vignere_Cipher.cwm
Home
Edit
Crypto Tutorials
New
Open
Save
Print
Play
Stop
Log
Startcenter
CrypToolStore
CrypCloud
Updates
Settings
Help
Components
Search
Classic Ciphers
ADFGVX
Affine Cipher
Bazeries Cipher
Bifid Cipher
Book Cipher
Caesar
Modern Ciphers
Steganography
Hash Functions
Cryptanalysis
Protocols
Tools
Vigenère
Mode
Vigenère Classic
Action
Encrypt
Key (integer)
1,2,3
Key (as string)
BCD
Alphabet parameters
Alphabet
ABCDEFGHIJKLMNOPQRSTUVWXYZ
Unknown symbol handling
Ignore (leave unmodified)
Case sensitive
Output contains Source Case
Hellow everyone
15 characters, 1 line
Text Input
0 characters, 0 lines
Text Output


CrypTool 2.1 (Stable Build 9778.2) – Practical1_Vignere_Cipher.cwm
Calculation finished after 0 seconds (To stop the workspace please push the stop button or enter new data to start a new calculation)
Hellow everyone
15 characters, 1 line
Text Input
IGOMQZ FXHSAROG
15 characters, 1 line
Text Output

6. Comparison

Feature	Caesar Cipher	Vigenère Cipher	Rail Fence Cipher
Technique	Substitution	Polyalphabetic substitution	Transposition
Key	Shift value	Keyword	Number of rails
Security	Very low	Low	Very low
Complexity	Simple	Moderate	Simple
Main weakness	Brute-force attack	Repeated-key patterns	Easy to break

7. Result

The Caesar, Vigenère, and Rail Fence ciphers were studied using CrypTool 2. Encryption and decryption procedures were demonstrated, and the expected ciphertexts were calculated. All three techniques are useful for understanding classical cryptography but are unsuitable for modern secure communication.

Practical 2: Frequency Analysis Using CrypTool 2

1. Aim

To perform frequency analysis on classical ciphertext using CrypTool 2 and understand the weaknesses of classical encryption techniques.

2. Software Required

- CrypTool 2
- Windows operating system
- Practical_2.cwm (provided in your ZIP)

3. Theory

Frequency analysis is a cryptanalysis technique used to study how frequently different letters appear in a ciphertext.

In English, certain letters such as **E, T, A, O, I, and N** appear more frequently than others.

By studying letter frequencies, an attacker may identify patterns and recover the original plaintext.

Frequency analysis is particularly effective against simple substitution ciphers because they preserve the statistical patterns of the original language.

Formula:

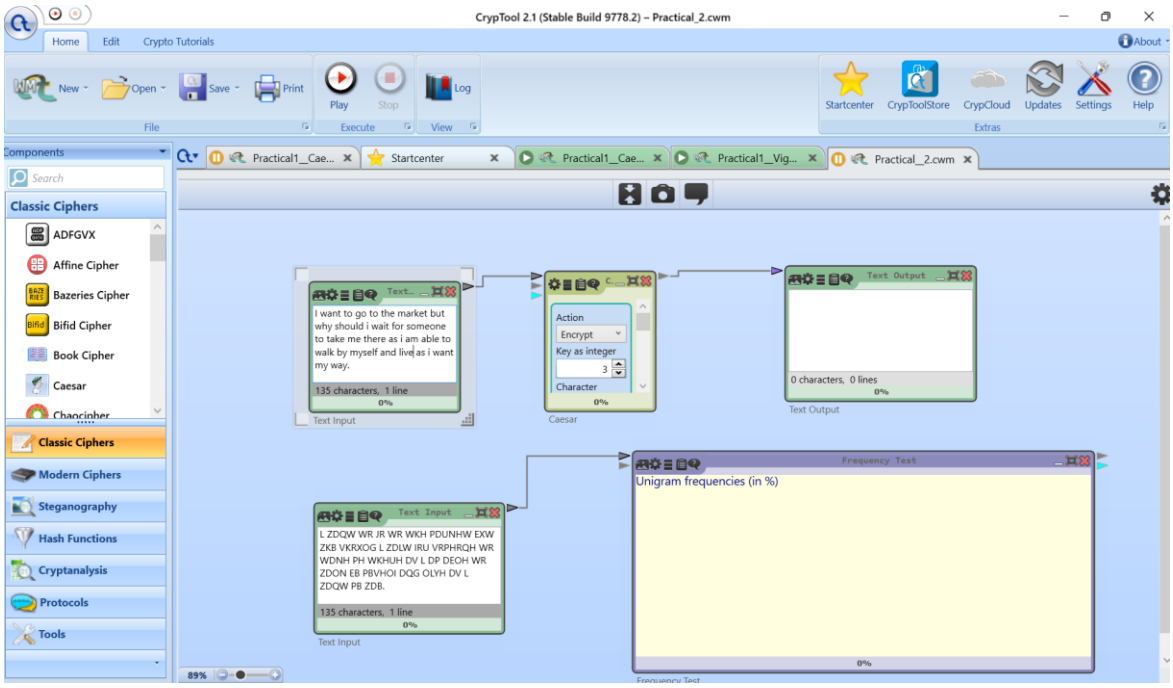
\[\text{Frequency (\%)}=\frac{\text{Occurrences of a letter}}{\text{Total number of letters}}\times100 \]

4. Algorithm

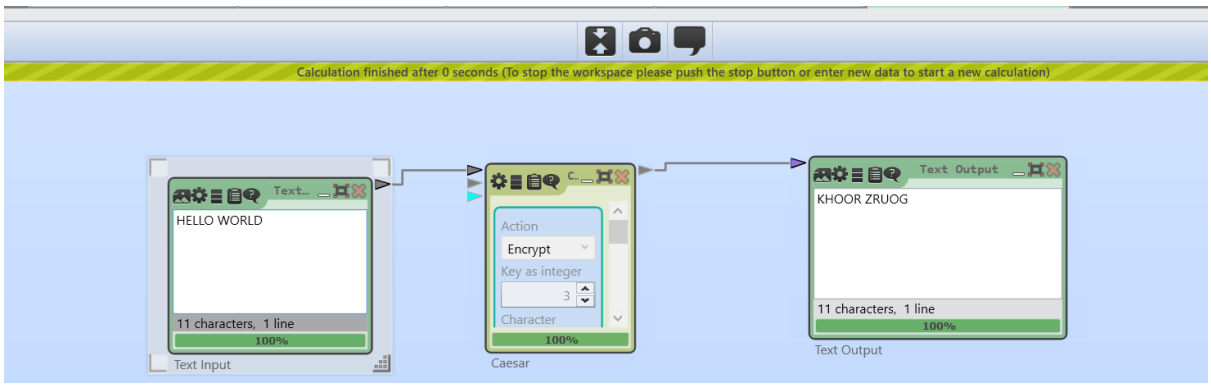
1. Start.
2. Open CrypTool 2.
3. Enter or load the ciphertext.
4. Select the Frequency Analysis component.
5. Execute the analysis.
6. Count the occurrences of each letter.
7. Display the frequency distribution.
8. Compare the distribution with standard English letter frequencies.
9. Identify possible plaintext letters.

10. Stop.

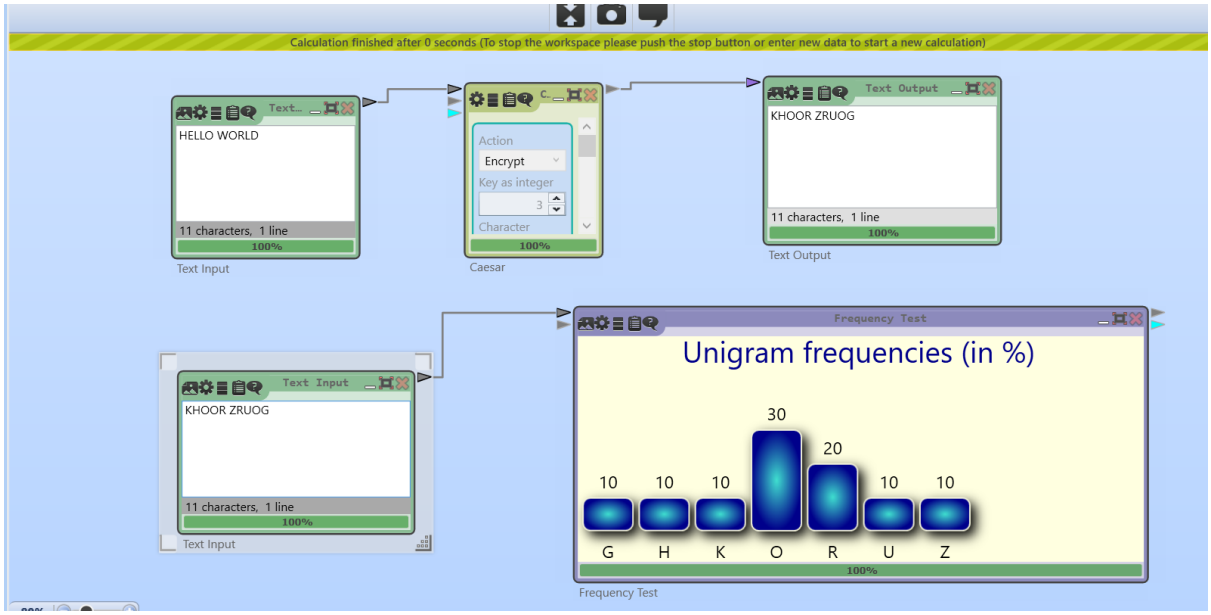
Screenshot 1 – Tool Setup



Screenshot 2: Enter the Input Text



Screenshot 3: Perform Frequency Analysis



Frequency Analysis:

Letter Count Frequency

O 3 30%

R 2 20%

K 1 10%

H 1 10%

Z 1 10%

U 1 10%

G 1 10%

Decrypted Text:

HELLO WORLD

Observation: The ciphertext contains repeated letters, which can provide clues about the original message. However, frequency analysis is less reliable for very short messages because they contain limited statistical information. In this example, frequency analysis alone does not uniquely determine the plaintext.

7. Result

Frequency analysis was studied using CrypTool 2. The frequency distribution of the ciphertext was examined, and the original plaintext was recovered by testing a Caesar shift. The experiment demonstrates that classical substitution ciphers are vulnerable to statistical analysis.

Practical 3: Implementation of DES and AES Using CrypTool 2

1. Aim

To implement DES (Data Encryption Standard) and AES (Advanced Encryption Standard) encryption and decryption using CrypTool 2 and compare their working and security.

2. Software Required

- CrypTool 2
- Windows Operating System
- PRACTICAL3_DES.cwm
- PRACTICAL3_AES.cwm

3. Theory

DES (Data Encryption Standard) is a symmetric-key encryption algorithm that uses the same key for encryption and decryption. It operates on 64-bit blocks with a 56-bit effective key and performs 16 rounds of encryption.

AES (Advanced Encryption Standard) is a modern symmetric-key encryption algorithm that operates on 128-bit blocks. It supports key sizes of 128, 192, and 256 bits and performs 10, 12, or 14 rounds, depending on the key size.

AES is more secure than DES because it uses larger encryption keys and a stronger encryption structure.

4. Algorithm

A. DES Algorithm

1. Start.
2. Enter the plaintext and encryption key.
3. Apply the initial permutation to the plaintext block.
4. Divide the block into left and right halves.
5. Perform 16 rounds of Feistel operations.
6. Apply the final permutation.
7. Generate the ciphertext.
8. Decrypt the ciphertext using the same key with the round keys in reverse order.

9. Stop.

B. AES Algorithm

1. Start.
2. Enter the plaintext and encryption key.
3. Perform the initial AddRoundKey operation.
4. Apply SubBytes, ShiftRows, MixColumns, and AddRoundKey transformations.
5. Repeat the transformations for the required rounds.
6. Perform the final round without MixColumns.
7. Generate the ciphertext.
8. Decrypt using the corresponding inverse transformations.
9. Stop.

5. Working Procedure

Part A: DES Encryption and Decryption

Step 1: Open CrypTool 2.

Step 2: Select File → Open and load PRACTICAL3_DES.cwm.

Step 3: Locate the plaintext input component and enter HELLO WORLD.

Step 4: Select the DES encryption component and configure the encryption key.

Step 5: Check the encryption mode, padding, and initialization vector (IV), if required.

Step 6: Connect the plaintext input, DES encryption component, and output component.

Step 7: Click the Play button to execute the encryption workflow.

Step 8: Configure the DES decryption component using the same key and matching settings.

Step 9: Connect the ciphertext output to the decryption component.

Step 10: Execute the workflow to perform decryption.

Part B: AES Encryption and Decryption

Step 1: Open CrypTool 2.

Step 2: Select File → Open and load PRACTICAL3_AES.cwm.

Step 3: Locate the plaintext input component and enter HELLO WORLD.

Step 4: Select the AES encryption component.

Step 5: Configure a valid AES encryption key of 128, 192, or 256 bits.

Step 6: Configure the encryption mode, padding, and IV, if required.

Step 7: Connect the plaintext input, AES encryption component, and output component.

Step 8: Click Play to execute the encryption workflow.

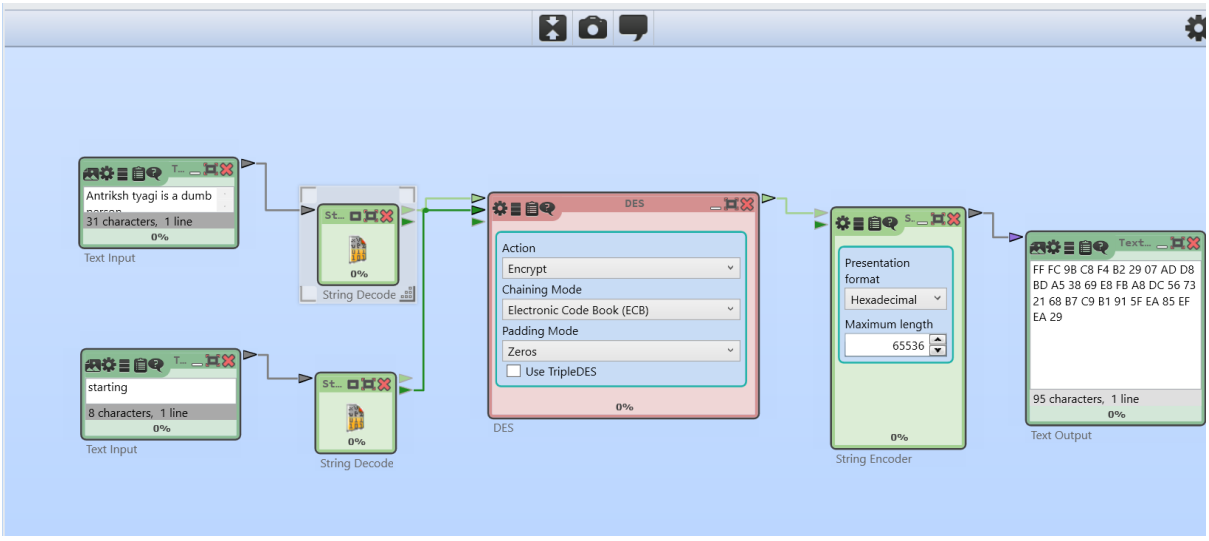
Step 9: Configure the AES decryption component using the same key and corresponding settings.

Step 10: Connect the ciphertext to the decryption component and execute the workflow.

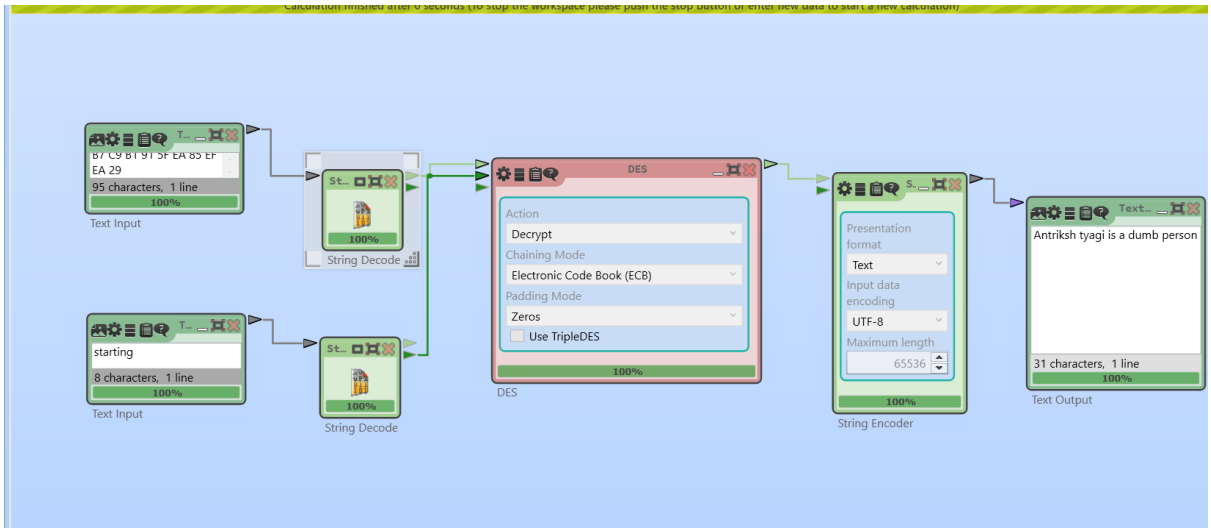
6. Output

A. DES Encryption and Decryption

Output: DES encryption result showing the generated ciphertext.

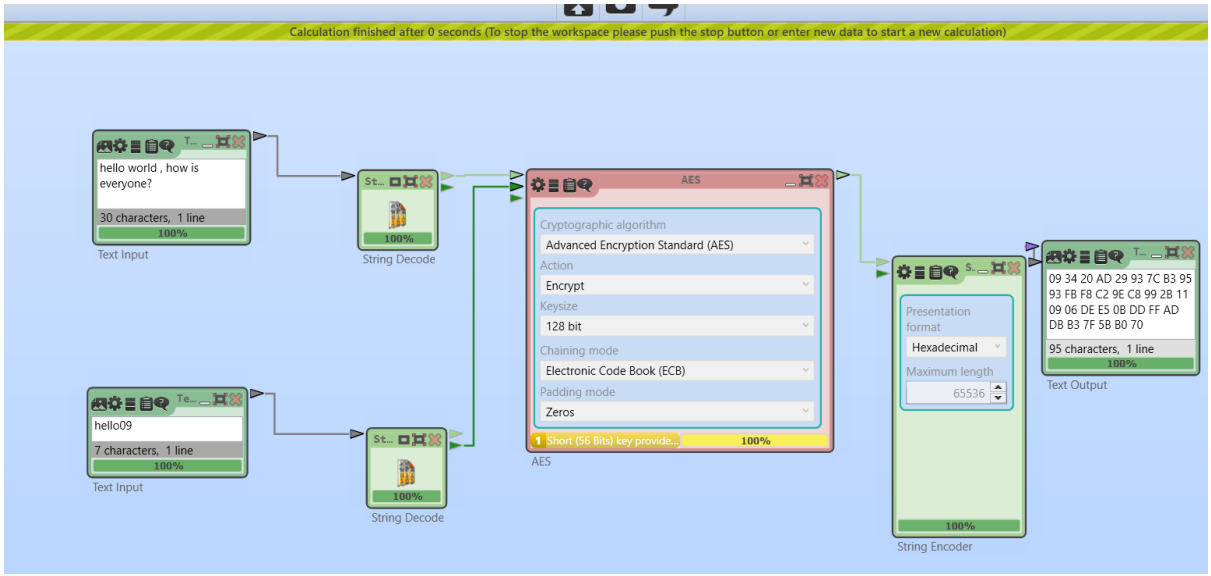


Output: DES decryption result showing the recovered plaintext.

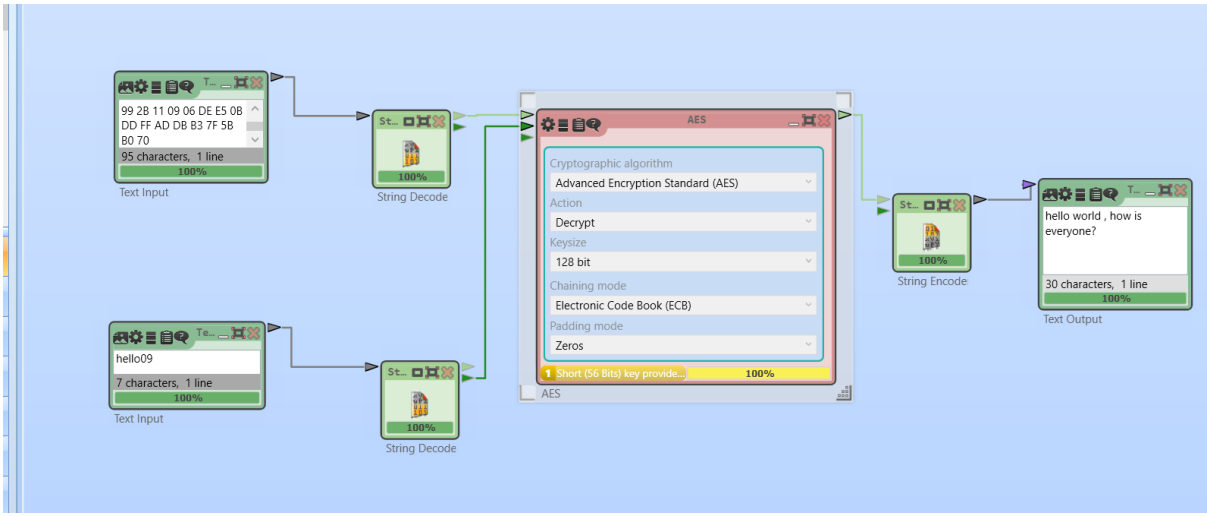


B. AES Encryption and Decryption

Output: AES encryption result showing the generated ciphertext.



Output: AES decryption result showing the recovered plaintext.



Note: The actual ciphertext depends on the encryption key, mode, IV, and padding settings used in CrypTool 2.

7. Comparison Between DES and AES

Feature	DES	AES
Algorithm Type	Symmetric	Symmetric
Block Size	64 bits	128 bits
Key Size	56 bits	128, 192, 256 bits
Number of Rounds	16	10, 12, 14
Security	Weak	Strong
Structure	Feistel Network	Substitution-Permutation Network
Current Usage	Obsolete	Widely used

8. Result

DES and AES encryption and decryption were studied using CrypTool 2. Both algorithms use symmetric-key cryptography, but AES provides stronger security and is preferred for modern applications.

Practical 4: RSA File Encryption and Decryption Using OpenSSL

1. Aim

To implement RSA encryption and decryption using OpenSSL by generating public and private keys, encrypting a text file, and recovering the original message.

2. Software Required

- OpenSSL
- Windows Operating System
- Command Prompt or PowerShell
- Text Editor

3. Theory

RSA (Rivest–Shamir–Adleman) is an asymmetric encryption algorithm that uses two different keys: a public key and a private key.

The **public key** is used for encryption, while the **private key** is used for decryption.

RSA security depends on the computational difficulty of factoring large integers.

Encryption Formula:

[
C=M^e\bmod n
]

Decryption Formula:

[
M=C^d\bmod n
]

Where:

- M = Plaintext represented as an integer
- C = Ciphertext
- e = Public exponent
- d = Private exponent
- n = Product of two large prime numbers

4. Algorithm

1. Start.

2. Generate two large prime numbers, p and q.
3. Calculate n = p × q.
4. Calculate ϕ(n) = (p − 1)(q − 1).
5. Choose a public exponent e relatively prime to ϕ(n).
6. Calculate the private exponent d.
7. Generate the public and private keys.
8. Encrypt the plaintext using the public key.
9. Decrypt the ciphertext using the private key.
10. Verify the recovered message.
11. Stop.

5. Working Procedure

Step 1: Open the Terminal

Open Command Prompt or PowerShell and navigate to the practical folder.

Step 2: Generate the RSA Private Key

Execute the following command:

openssl genpkey -algorithm RSA -out private.pem -pkeyopt rsa_keygen_bits:2048

This command generates a 2048-bit RSA private key.

Step 3: Generate the Public Key

Execute:

openssl pkey -in private.pem -pubout -out public.pem

This command extracts the public key from the private key.

Step 4: Create the Plaintext File

Create a text file named message.txt using any text editor and enter the message to be encrypted.

Save the file in the practical folder.

Step 5: Encrypt the File

Execute:

openssl pkeyutl -encrypt -pubin -inkey public.pem -in message.txt -out encrypted.bin -pkeyopt rsa_padding_mode:oaep

This command encrypts the plaintext using the RSA public key.

Step 6: Decrypt the File

Execute:

openssl pkeyutl -decrypt -inkey private.pem -in encrypted.bin -out decrypted.txt -pkeyopt rsa_padding_mode:oaep

This command decrypts the encrypted file using the RSA private key.

Step 7: Verify the Decrypted File

Execute:

Get-Content decrypted.txt

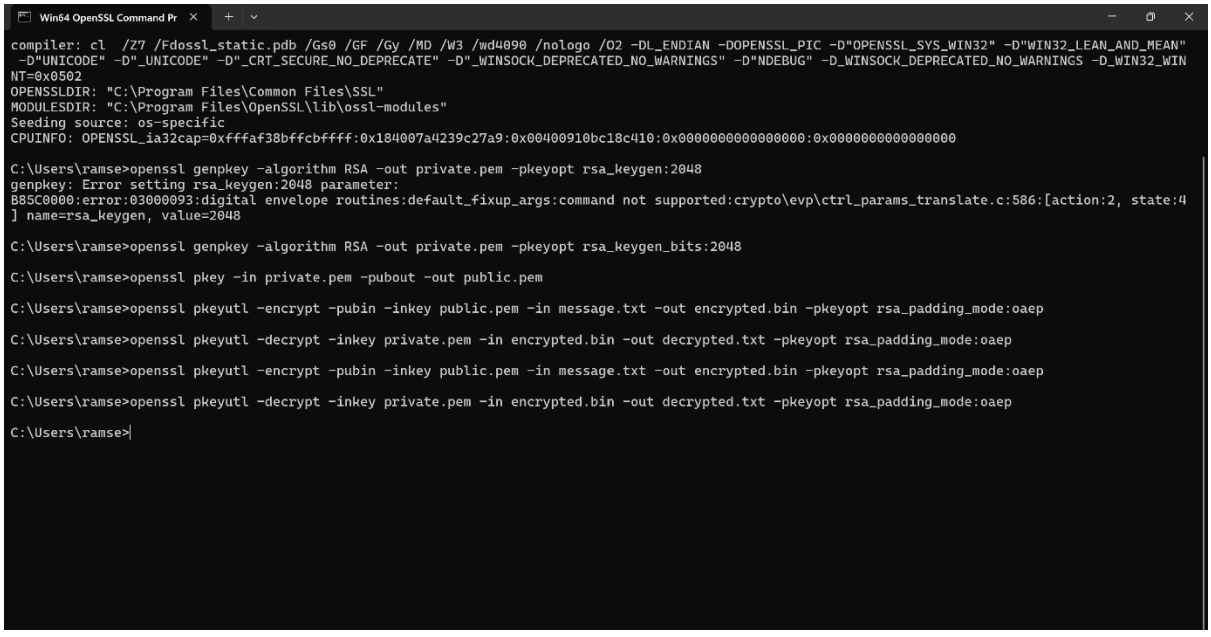
Compare the decrypted message with the original plaintext.

Setup Screenshots

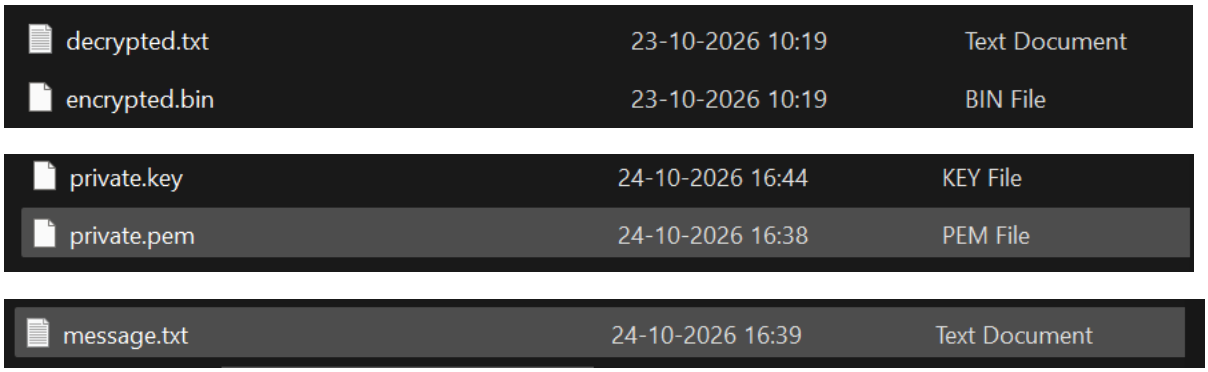
Screenshot 1: RSA private and public key generation.

Screenshot 2: RSA encryption and decryption commands.

Output:



Output:



7. Observation

RSA uses separate keys for encryption and decryption. The public key encrypts the plaintext, while the corresponding private key decrypts it.

RSA provides secure asymmetric encryption when suitable key sizes and padding methods are used. It is generally used to protect small amounts of data rather than large files directly.

8. Result

RSA encryption and decryption were implemented using OpenSSL. The experiment demonstrated the working of asymmetric-key cryptography using public and private keys.

Practical 5: Implementation of SHA-256 Hashing and Digital Signatures Using OpenSSL

1. Aim

To implement SHA-256 hashing and RSA digital signatures using OpenSSL to verify data integrity, authenticity, and detect unauthorized modifications.

2. Software Required

- OpenSSL
- Windows Operating System
- Command Prompt or PowerShell
- Text Editor

3. Theory

SHA-256 (Secure Hash Algorithm 256-bit) is a cryptographic hash function that converts input data into a fixed-length 256-bit hash value.

It is a one-way function, meaning the original message cannot practically be recovered from its hash value.

Digital Signature is a cryptographic technique used to verify the authenticity and integrity of digital information.

Digital signatures use asymmetric cryptography, where the private key is used to generate a signature and the corresponding public key is used to verify it.

Digital signatures provide:

- **Authentication:** Confirms the identity of the signer.
- **Integrity:** Ensures that the message has not been modified.
- **Non-repudiation:** Provides evidence that the holder of the private key signed the message.

4. Algorithm

A. SHA-256 Hashing

1. Start.
2. Enter or create the input message.
3. Convert the message into binary form.
4. Apply SHA-256 padding.

5. Divide the padded message into 512-bit blocks.
6. Process each block using SHA-256 compression operations.
7. Generate the final 256-bit hash value.
8. Stop.

B. RSA Digital Signature

1. Start.
2. Generate an RSA private and public key pair.
3. Create a message file.
4. Calculate the SHA-256 hash of the message.
5. Generate a digital signature using the private key.
6. Save the signature in a separate file.
7. Verify the signature using the public key.
8. Check whether the message has been modified.
9. Stop.

5. Working Procedure

Step 1: Open the Terminal

Open Command Prompt or PowerShell and navigate to the practical folder.

Step 2: Create a Message File

Create a text file named message.txt using a text editor.

Enter the message to be signed and save the file.

Step 3: Generate the SHA-256 Hash

Execute the following command:

openssl dgst -sha256 message.txt

This command calculates the SHA-256 hash of the message file.

Step 4: Generate the RSA Private Key

Execute:

openssl genpkey -algorithm RSA -out private.pem -pkeyopt rsa_keygen_bits:2048

This command generates a 2048-bit RSA private key.

Step 5: Generate the RSA Public Key

Execute:

openssl pkey -in private.pem -pubout -out public.pem

This command extracts the public key from the private key.

Step 6: Generate the Digital Signature

Execute:

openssl dgst -sha256 -sign private.pem -out signature.bin message.txt

This command calculates the SHA-256 hash and generates an RSA digital signature using the private key.

Step 7: Verify the Digital Signature

Execute:

openssl dgst -sha256 -verify public.pem -signature signature.bin message.txt

This command verifies the digital signature using the public key.

Step 8: Test Message Integrity

Modify the contents of message.txt and save the file.

Execute the verification command again:

openssl dgst -sha256 -verify public.pem -signature signature.bin message.txt

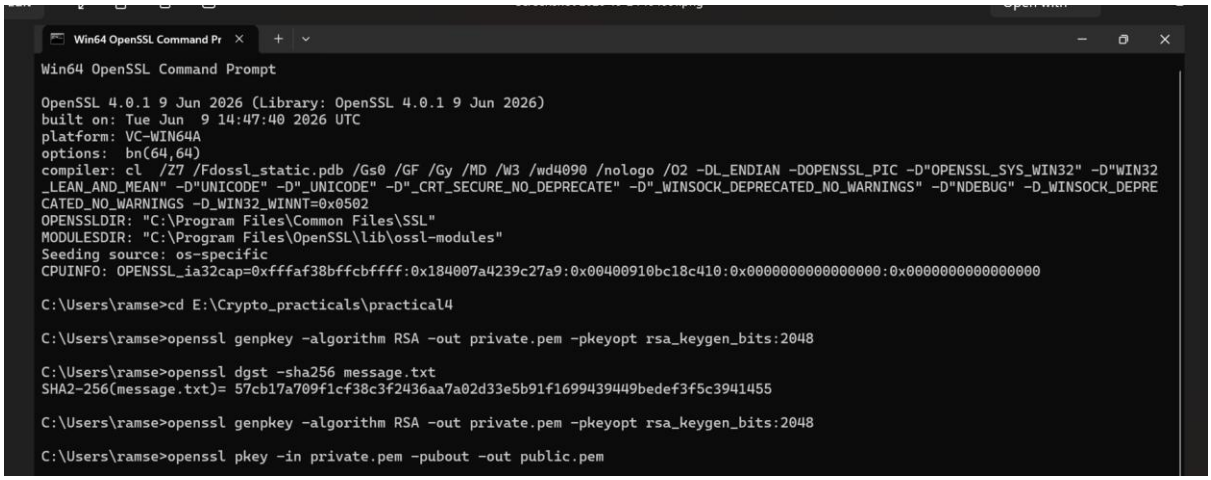
The verification process checks whether the message matches the original signed data.

Step 9: Restore the Original Message

Restore the original contents of message.txt and save the file.

Repeat the signature verification command to check the original message.

Output:



Output:



7. Observation

SHA-256 generates a fixed-length hash value for the input message. Even a small modification in the message produces a significantly different hash.

RSA digital signatures use the private key for signing and the public key for verification.

If the signed message is modified, signature verification fails, demonstrating the importance of digital signatures in maintaining data integrity and authenticity.

8. Result

SHA-256 hashing and RSA digital signature generation and verification were implemented using OpenSSL. The experiment demonstrated how cryptographic hashing and digital signatures help ensure data integrity and authenticity.

Practical 6: X.509 Digital Certificate Generation and Verification Using OpenSSL

1. Aim

To generate and verify an X.509 digital certificate using OpenSSL and understand its role in authentication and secure communication.

2. Software Required

- OpenSSL
- Windows Operating System
- Command Prompt or PowerShell
- Text Editor

3. Theory

An **X.509 digital certificate** is an electronic document used to verify the identity of a person, organization, or server.

It contains information such as the certificate holder's identity, public key, issuer details, validity period, and digital signature.

X.509 certificates are commonly used in SSL/TLS communication to establish secure connections.

Main Components of an X.509 Certificate:

- **Subject:** Identity of the certificate holder.
- **Issuer:** Entity that issued the certificate.
- **Public Key:** Used for cryptographic operations.
- **Serial Number:** Unique identifier assigned by the issuer.
- **Validity Period:** Specifies when the certificate is valid.
- **Digital Signature:** Helps verify the certificate's authenticity and integrity.

A **Certificate Authority (CA)** is a trusted organization responsible for issuing and signing digital certificates.

A self-signed certificate is signed using its own private key rather than by an external CA.

4. Algorithm

1. Start.
2. Open the OpenSSL terminal.
3. Generate an RSA private key.
4. Create a Certificate Signing Request (CSR).
5. Enter the required certificate information.
6. Generate a self-signed X.509 certificate.
7. Display the certificate details.
8. Verify the certificate using OpenSSL.
9. Examine the certificate validity and signature.
10. Stop.

5. Working Procedure

Step 1: Open the Terminal

Open Command Prompt or PowerShell and navigate to the practical folder.

Step 2: Generate the RSA Private Key

Execute the following command:

openssl genpkey -algorithm RSA -out private.key -pkeyopt rsa_keygen_bits:2048

This command generates a 2048-bit RSA private key.

Step 3: Generate a Certificate Signing Request

Execute:

openssl req -new -key private.key -out request.csr

Enter the required information, such as country, state, organization, and common name.

The CSR contains the public key and identity information required for certificate generation.

Step 4: Generate a Self-Signed X.509 Certificate

Execute:

openssl x509 -req -in request.csr -signkey private.key -out certificate.crt -days 365 -sha256

This command generates a self-signed X.509 certificate valid for 365 days.

Step 5: Display Certificate Details

Execute:

openssl x509 -in certificate.crt -text -noout

This command displays the certificate information, including the issuer, subject, public key, validity period, and signature algorithm.

Step 6: Verify the Certificate

Execute:

openssl verify -CAfile certificate.crt certificate.crt

This command verifies the self-signed certificate using the certificate itself as the trusted certificate.

Step 7: Check Certificate Validity

Execute:

openssl x509 -in certificate.crt -noout -dates

This command displays the certificate's starting and expiration dates.

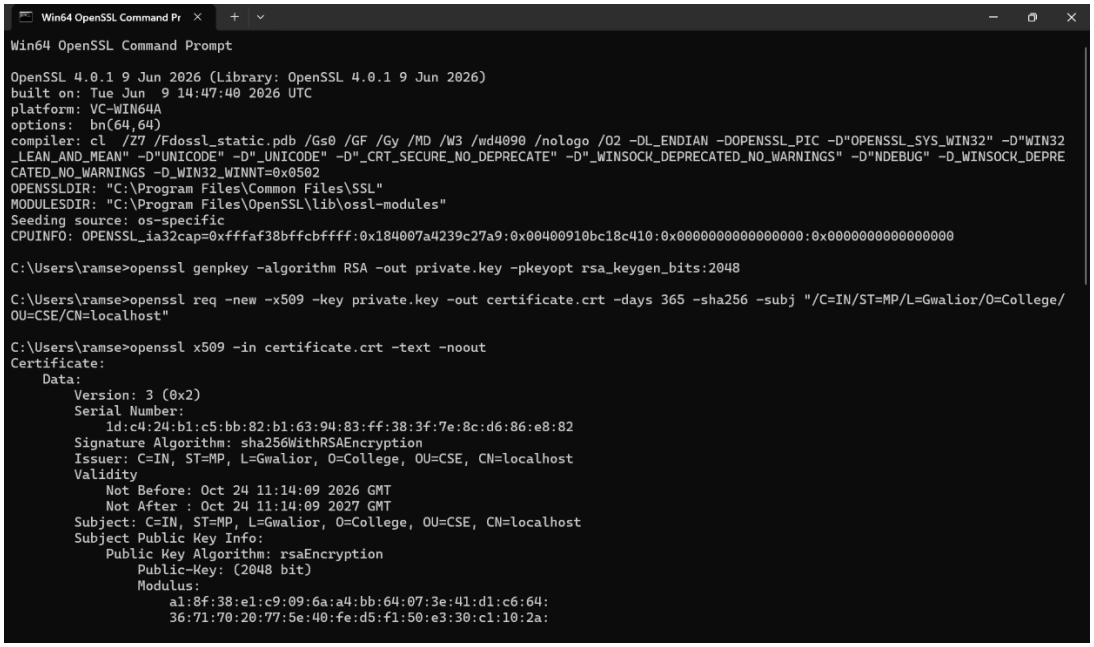
Step 8: Examine the Certificate

Open the generated certificate.crt file using a text editor or certificate viewer.

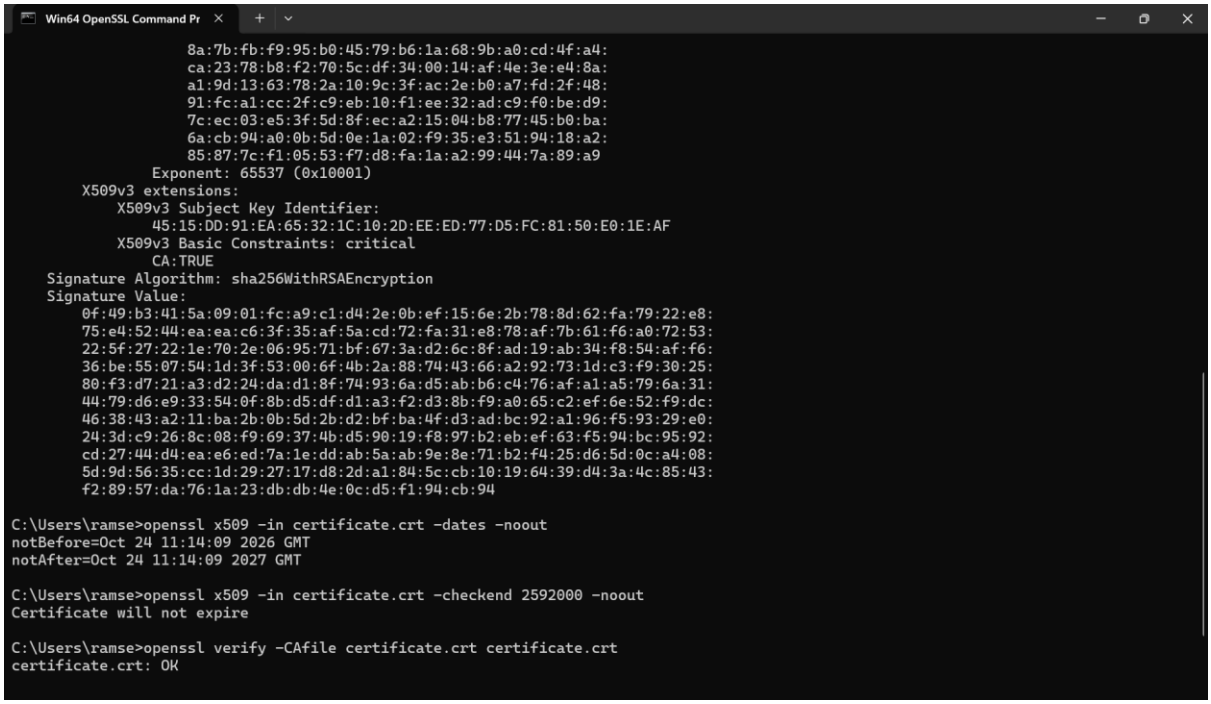
Examine the certificate information and verify that the required details are present.

verification and validity-checking commands.

Output:



Output:



7. Observation

The X.509 certificate contains the public key, subject information, issuer information, validity period, and digital signature.

A self-signed certificate is issued and signed by the same entity. Its signature can be verified, but it is not automatically trusted by external systems.

X.509 certificates play an important role in authentication and secure communication.

8. Result

An X.509 digital certificate was generated and verified using OpenSSL. The experiment demonstrated the process of certificate creation, verification, and its importance in establishing secure communication.

Practical 7: Implementation of RSA Algorithm Using Python

1. Aim

To implement the RSA algorithm using Python for public-key generation, encryption, and decryption of a message.

2. Software Required

- Python 3
- Visual Studio Code or Python IDLE
- Windows Operating System

3. Theory

RSA (Rivest–Shamir–Adleman) is an asymmetric cryptographic algorithm that uses two different keys: a public key and a private key.

The public key is used for encryption, while the private key is used for decryption.

RSA is based on mathematical operations involving large prime numbers and modular arithmetic.

Key Generation:

Two prime numbers, p and q, are selected.

[
n=p\times q
]

Euler's totient function is calculated as:

[
\phi(n)=(p-1)(q-1)
]

The public exponent e is selected such that:

[
\gcd(e,\phi(n))=1
]

The private exponent d is calculated as:

[
d=e^{-1}\bmod\phi(n)
]

Encryption Formula:

[
C=M^e\bmod n
]

Decryption Formula:

[
M=C^d\bmod n
]

Where M is the plaintext, C is the ciphertext, e is the public exponent, d is the private exponent, and n is the modulus.

4. Algorithm

1. Start.
2. Select two prime numbers, p and q.
3. Calculate n = p × q.
4. Calculate ϕ(n) = (p − 1)(q − 1).
5. Select a public exponent e.
6. Calculate the private exponent d using modular inverse.
7. Generate the public and private keys.
8. Enter the message to be encrypted.
9. Encrypt the message using the public key.
10. Decrypt the ciphertext using the private key.
11. Display the encrypted and decrypted messages.
12. Stop.

5. Working Procedure

Step 1: Open Python Environment

Open Visual Studio Code or Python IDLE.

Create a new Python file named practical_7.py.

Step 2: Write the RSA Program

Enter the following Python code:

```
# RSA Algorithm

p = 61

q = 53

n = p * q

phi = (p - 1) * (q - 1)

e = 17

d = pow(e, -1, phi)

print("Public Key:", (e, n))

print("Private Key:", (d, n))

message = int(input("Enter a number to encrypt: "))

encrypted = pow(message, e, n)

print("Encrypted Message:", encrypted)

decrypted = pow(encrypted, d, n)

print("Decrypted Message:", decrypted)
```

Step 3: Save the Program

Save the file as practical_7.py.

Step 4: Execute the Program

Open the terminal in Visual Studio Code.

Run the following command:

python practical_7.py

Step 5: Enter the Message

Enter an integer message when prompted.

The message must satisfy:

[
0\leq M<n
]

Step 6: Perform Encryption

The program uses the public key to encrypt the entered message through modular exponentiation.

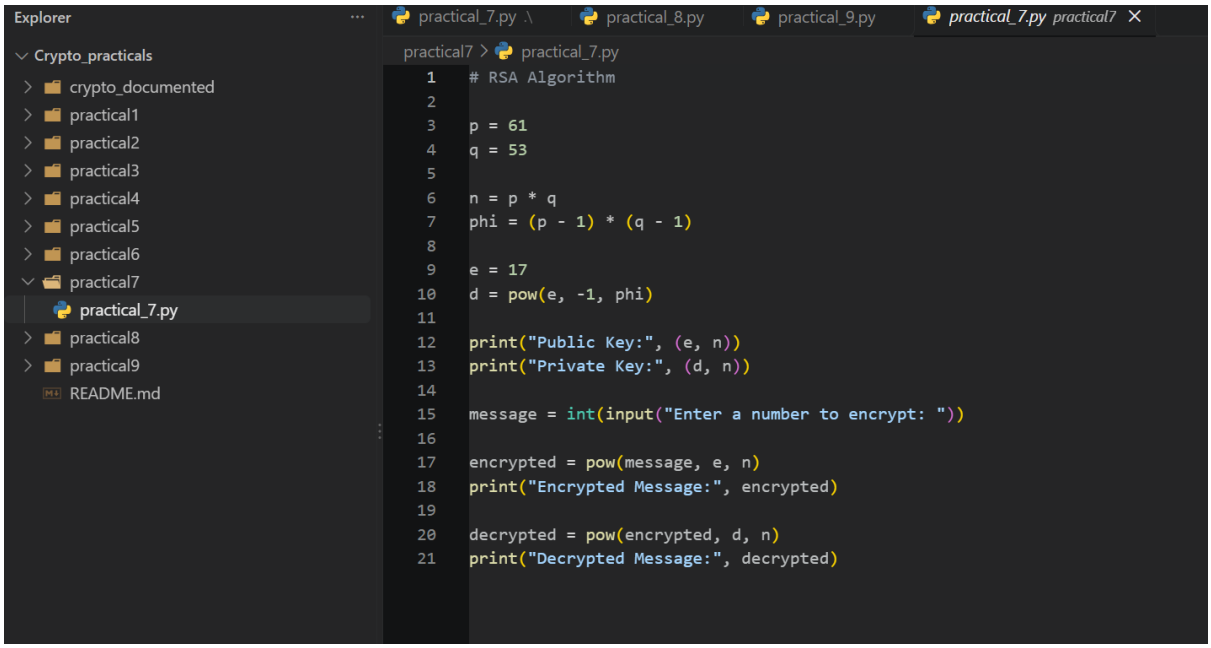
Step 7: Perform Decryption

The program uses the private key to decrypt the ciphertext and recover the original integer message.

Step 8: Verify the Message

Compare the decrypted message with the original input to verify successful encryption and decryption.

Output:



Output :

```
PS E:\Crypto_practicals> & "C:\Program Files\Python313\python.exe" e:/Crypto_practicals/practical7/practical_7.py
Public Key: (17, 3233)
Private Key: (2753, 3233)
Enter a number to encrypt: 67
Encrypted Message: 641
Decrypted Message: 67
PS E:\Crypto_practicals>
```

7. Observation

RSA uses public and private keys for encryption and decryption.

The public key is generated using the public exponent and modulus, while the private key is calculated using the modular inverse.

The encrypted message is converted into ciphertext, and the original message is recovered using the private key.

The program demonstrates the basic mathematical working of RSA using small prime numbers for educational purposes.

8. Result

The RSA algorithm was implemented using Python. Public and private keys were generated, and encryption and decryption operations were performed to demonstrate asymmetric-key cryptography.

Practical 8: Implementation of AES Encryption and SHA-256 Hashing Using Python

1. Aim

To implement AES encryption and decryption along with SHA-256 hashing using Python to ensure message confidentiality and verify data integrity.

2. Software Required

- Python 3
- Visual Studio Code or Python IDLE
- Cryptography Library
- Hashlib Library
- Windows Operating System

3. Theory

AES (Advanced Encryption Standard) is a symmetric-key encryption algorithm that uses the same secret key for encryption and decryption.

AES operates on 128-bit blocks and supports key sizes of 128, 192, and 256 bits.

In this experiment, AES-GCM (Galois/Counter Mode) is used. It provides both encryption and authentication, helping protect messages against unauthorized modification.

SHA-256 (Secure Hash Algorithm 256-bit) is a cryptographic hash function that converts input data into a fixed-length 256-bit hash value.

SHA-256 is a one-way function, meaning the original message cannot practically be recovered from its hash.

Main Differences:

- AES is used to encrypt and decrypt information.
- SHA-256 is used to generate a hash value.
- AES requires a secret key.
- SHA-256 does not require an encryption key.
- AES-GCM provides authenticated encryption, while SHA-256 alone does not authenticate the sender.

4. Algorithm

1. Start.

2. Import the required Python libraries.
3. Accept a message from the user.
4. Generate a random 256-bit AES key.
5. Generate a random nonce.
6. Encrypt the message using AES-GCM.
7. Generate the SHA-256 hash of the original message.
8. Decrypt the ciphertext using the same AES key and nonce.
9. Calculate the SHA-256 hash of the decrypted message.
10. Compare both hash values.
11. Verify message integrity.
12. Stop.

5. Working Procedure

Step 1: Open Python Environment

Open Visual Studio Code or Python IDLE.

Navigate to the practical8 folder.

Step 2: Install the Required Library

Open the terminal and execute:

pip install cryptography

The hashlib and os libraries are included with Python.

Step 3: Open the Python Program

Open the existing file named practical_8.py.

Step 4: Write the Program

Enter the following Python code:

```
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
import hashlib
import os


# Enter message
```

```
message = input("Enter your message: ")

# Generate AES key
key = AESGCM.generate_key(bit_length=256)
aes = AESGCM(key)

# Generate random nonce
nonce = os.urandom(12)

# Encrypt message
encrypted = aes.encrypt(nonce, message.encode(), None)

print("\nEncrypted Message:", encrypted.hex())

# Generate SHA-256 hash
hash_value = hashlib.sha256(message.encode()).hexdigest()

print("SHA-256 Hash:", hash_value)

# Decrypt message
decrypted = aes.decrypt(nonce, encrypted, None).decode()

print("Decrypted Message:", decrypted)

# Verify hash
new_hash = hashlib.sha256(decrypted.encode()).hexdigest()

if hash_value == new_hash:
```

```
    print("Message Integrity Verified!")
else:
    print("Message Integrity Failed!")
```

Step 5: Save the Program

Save the file as practical_8.py.

Step 6: Execute the Program

Open the VS Code terminal and execute:

python practical_8.py

Step 7: Enter the Message

Enter the text message when prompted by the program.

Step 8: Perform AES Encryption

The program generates a random AES key and nonce.

It encrypts the entered message using AES-GCM.

Step 9: Generate the SHA-256 Hash

The program calculates the SHA-256 hash of the original message using the hashlib library.

Step 10: Perform AES Decryption

The encrypted message is decrypted using the same AES key and nonce.

Step 11: Verify Message Integrity

The program calculates the SHA-256 hash of the decrypted message and compares it with the original hash.

This comparison checks whether the decrypted message matches the original message.

Output:

practical_7.py practical_8.py .\ practical_9.py practical_8.py practical8

practical8 > practical_8.py > ...

```python
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
import hashlib
import os

# Enter message
message = input("Enter your message: ")

# Generate AES key
key = AESGCM.generate_key(bit_length=256)
aes = AESGCM(key)

# Generate random nonce
nonce = os.urandom(12)

# Encrypt message
encrypted = aes.encrypt(nonce, message.encode(), None)

print("\nEncrypted Message:", encrypted.hex())

# Generate SHA-256 hash
hash_value = hashlib.sha256(message.encode()).hexdigest()

print("SHA-256 Hash:", hash_value)

# Decrypt message
decrypted = aes.decrypt(nonce, encrypted, None).decode()

print("Decrypted Message:", decrypted)

# Verify hash
new_hash = hashlib.sha256(decrypted.encode()).hexdigest()

if hash_value == new_hash:
    print("Message Integrity Verified!")
else:
    print("Message Integrity Failed!")
```

Output:

```
PS E:\Crypto_practicals> & "C:\Program Files\Python313\python.exe" e:/Crypto_practicals/practical8/practical_8.py
Enter your message: hi world

Encrypted Message: 3e15029f2def77dd19593bdbc1a26157d3b17cf94ac96246
SHA-256 Hash: c1a6857fc371e1eca52bdb3f4eb6982f6e9cc92e634365b64c1d162501764388
Decrypted Message: hi world
Message Integrity Verified!
PS E:\Crypto_practicals>
```

7. Observation

AES-GCM encrypts the message using a randomly generated 256-bit key and nonce.

SHA-256 generates a fixed-length hash value for the original message.

The decrypted message can be compared with the original message using their SHA-256 hashes.

AES-GCM also provides authentication through an authentication tag, allowing it to detect modifications to encrypted data during decryption.

8. Result

AES encryption and decryption, along with SHA-256 hashing, were implemented using Python. The experiment demonstrated message confidentiality through encryption and data integrity verification through hashing.

Practical 9: Implementation of Password Hashing With and Without Salt Using Python

1. Aim

To implement password hashing with and without salt using SHA-256 in Python and demonstrate the effect of salting on dictionary attacks.

2. Software Required

- Python 3
- Visual Studio Code or Python IDLE
- Hashlib Library
- OS Library
- Windows Operating System

3. Theory

Password Hashing is a technique used to convert a password into a fixed-length hash value using a cryptographic hash function.

Hashing is a one-way process, meaning the original password cannot practically be recovered directly from its hash.

SHA-256 (Secure Hash Algorithm 256-bit) generates a 256-bit hash value for the given input.

Salting is the process of adding a randomly generated value, called a salt, to a password before hashing.

Salting ensures that identical passwords can produce different hash values when different salts are used.

Dictionary Attack is a password-cracking technique in which an attacker tries commonly used passwords and compares their calculated hashes with a target hash.

Salting helps prevent precomputed hash attacks, but it does not prevent attackers from guessing weak passwords when the salt is known.

For real password storage, specialized password-hashing algorithms such as Argon2id, bcrypt, or scrypt are preferred over SHA-256.

4. Algorithm

1. Start.
2. Import the required Python libraries.

3. Accept a password from the user.
4. Generate the SHA-256 hash of the password without salt.
5. Generate a random 16-byte salt.
6. Combine the salt with the password.
7. Generate the SHA-256 hash of the salted password.
8. Create a list of commonly used passwords.
9. Compare the hash of each guessed password with the unsalted hash.
10. Repeat the dictionary attack using the same salt.
11. Display the comparison results.
12. Stop.

5. Working Procedure

Step 1: Open Python Environment

Open Visual Studio Code or Python IDLE.

Navigate to the practical9 folder.

Step 2: Open the Python File

Open the existing Python program named practical_9.py.

Step 3: Write the Program

Enter the following Python code:

```
import hashlib
import os

password = input("Enter Password: ")

# 1. Hashing without salt
hash1 = hashlib.sha256(password.encode()).hexdigest()
print("\nHash Without Salt:", hash1)

# 2. Hashing with salt
```

```
salt = os.urandom(16)

hash2 = hashlib.sha256(salt + password.encode()).hexdigest()

print("\nSalt:", salt.hex())
print("Hash With Salt:", hash2)

# 3. Dictionary Attack Simulation
common_passwords = ["123456", "password", "admin", "hello123"]

print("\nDictionary Attack (Without Salt):")

found = False

for word in common_passwords:
    test_hash = hashlib.sha256(word.encode()).hexdigest()

    if test_hash == hash1:
        print("Password Cracked:", word)
        found = True
        break

if not found:
    print("Password Not Found")

# 4. Dictionary Attack With Salt
print("\nDictionary Attack (With Salt):")
```

```
found = False


for word in common_passwords:
    test_hash = hashlib.sha256(salt + word.encode()).hexdigest()


    if test_hash == hash2:
        print("Password Cracked:", word)
        found = True
        break


if not found:
    print("Password Not Found")
```

Step 4: Save the Program

Save the file as practical_9.py.

Step 5: Execute the Program

Open the terminal in Visual Studio Code.

Run the following command:

python practical_9.py

Step 6: Enter the Password

Enter a password when prompted by the program.

The program processes the entered password using SHA-256.

Step 7: Perform Hashing Without Salt

The program generates a SHA-256 hash directly from the entered password using the hashlib library.

Step 8: Perform Hashing With Salt

The program generates a random 16-byte salt using the os library.

It combines the salt with the password and calculates a new SHA-256 hash.

Step 9: Perform Dictionary Attack Without Salt

The program checks a list of commonly used passwords.

Each guessed password is hashed and compared with the original unsalted hash.

Step 10: Perform Dictionary Attack With Salt

The program repeats the dictionary attack by combining each guessed password with the generated salt.

The resulting hashes are compared with the salted password hash.

Step 11: Compare the Results

Observe the differences between salted and unsalted hashes.

Compare the dictionary attack results to understand how salting affects password security.

Output:



Output:

```
PS E:\Crypto_practicals> & "C:\Program Files\Python313\python.exe" e:/Crypto_practicals/practical9/practical_9.py
Enter Password: admin

Hash Without Salt: 8c6976e5b5410415bde908bd4dee15dfb167a9c873fc4bb8a81f6f2ab448a918

Salt: 610513d15fba5fd2ab3fec2051a63f03
Hash With Salt: cc443592113514e3d0f2e5924faea32399a28ad2f63ba22e03d6789a7256bd2d

Dictionary Attack (Without Salt):
Password Cracked: admin

Dictionary Attack (With Salt):
Password Cracked: admin
PS E:\Crypto_practicals>
```

7. Observation

SHA-256 generates a fixed-length hash value for the entered password.

Without salting, identical passwords always generate identical SHA-256 hashes.

With salting, a random value is added before hashing, producing different hashes for the same password when different salts are used.

The dictionary attack demonstrates that weak passwords can still be guessed even when salting is applied.

Salting improves password storage security by preventing the direct reuse of precomputed password hashes.

8. Result

Password hashing with and without salt was implemented using Python. Dictionary attack simulations were performed to understand the limitations of basic hashing and the importance of salting in password security.